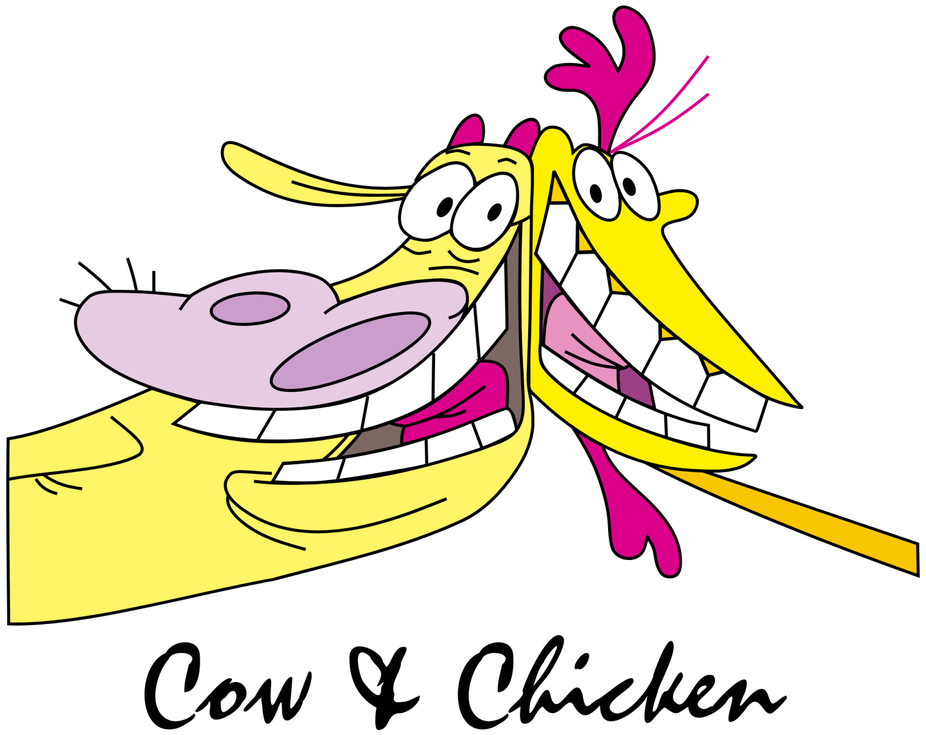
法海



如何设计微前端中的主子路由调度

论如何用大篇幅讲一个小问题

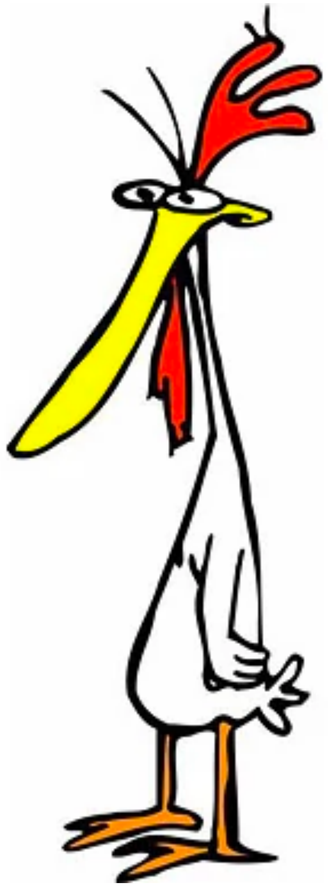
关于分享人

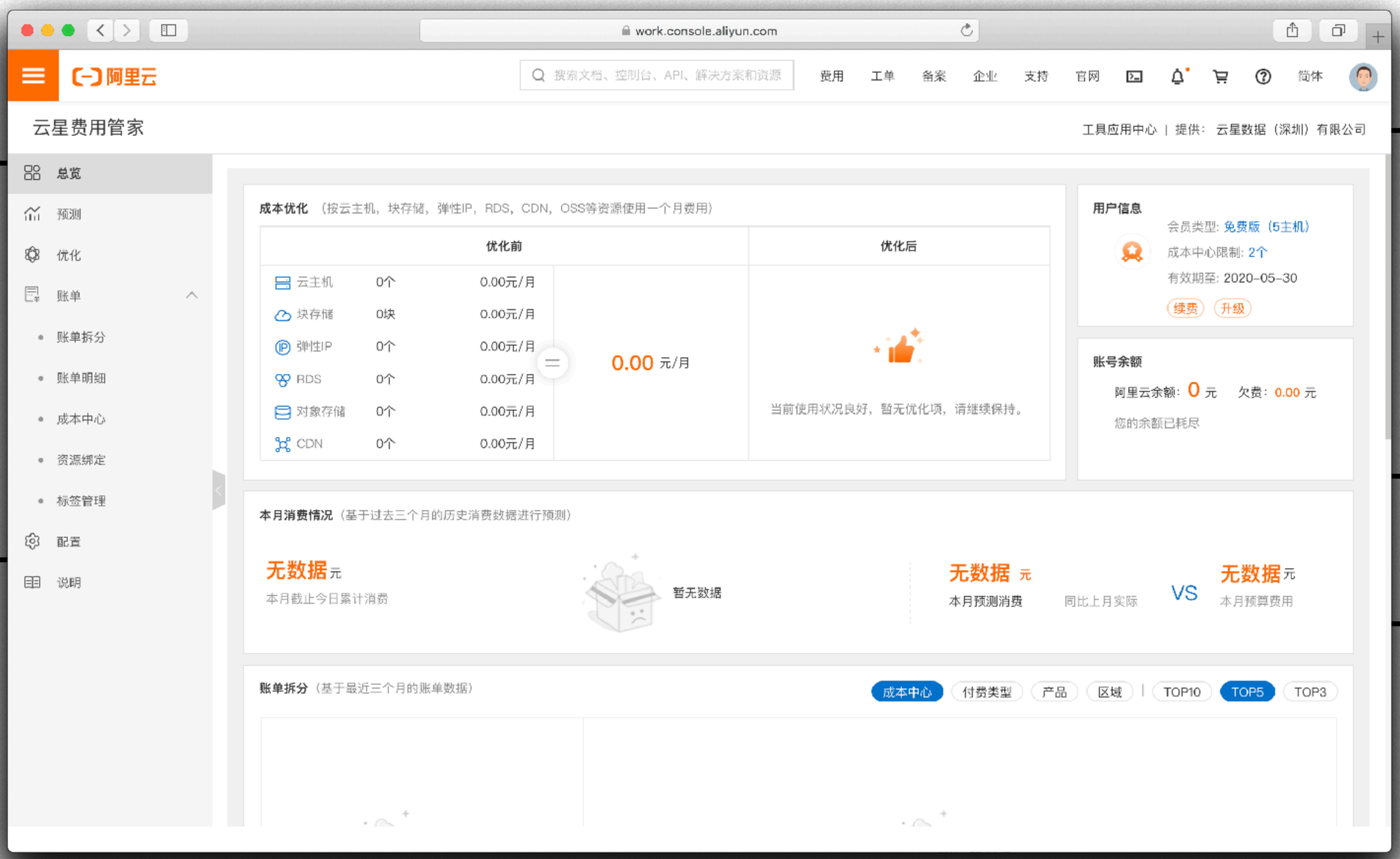
- 软件工程 -> 前端
- 淘系各种团队 -> 阿里云开放平台团队
- 即将被阿里向社会输送
- 亟需新鲜血液涌入！



关于分享内容

1. 背景：微前端在阿里云开放平台
2. 微前端技术体系全貌
3. 微前端集成带来的难题：路由调度
    1. 主应用完全不处理路由
    2. 子应用和主应用共享路由
    3. 子应用维护隔离的路由
    4. 子应用路由同步回主应用
4. 结语





真 · 巨石

微服务
原子化的
OpenAPI

三方系统集成

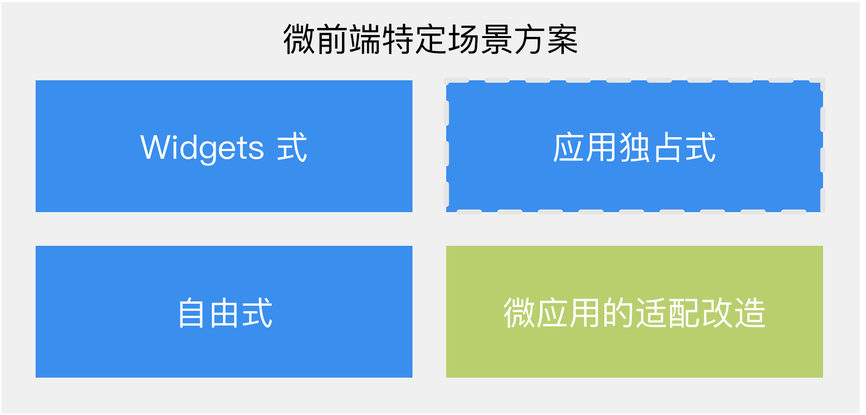
较细粒度的分治

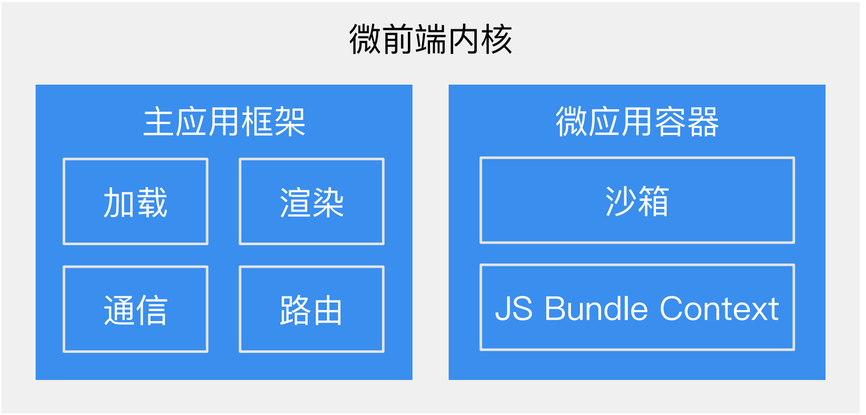
跨控制台模块调用

和而不同

不同的上层场景，相同的内核

- 独占 or 并存?
- 单实例 or 多实例?
- 隔离程度高 or 低?
- 占据的 UI 面积大 or 小?
- 技术栈相同 or 不同?
- 对路由有依赖 or 无依赖?

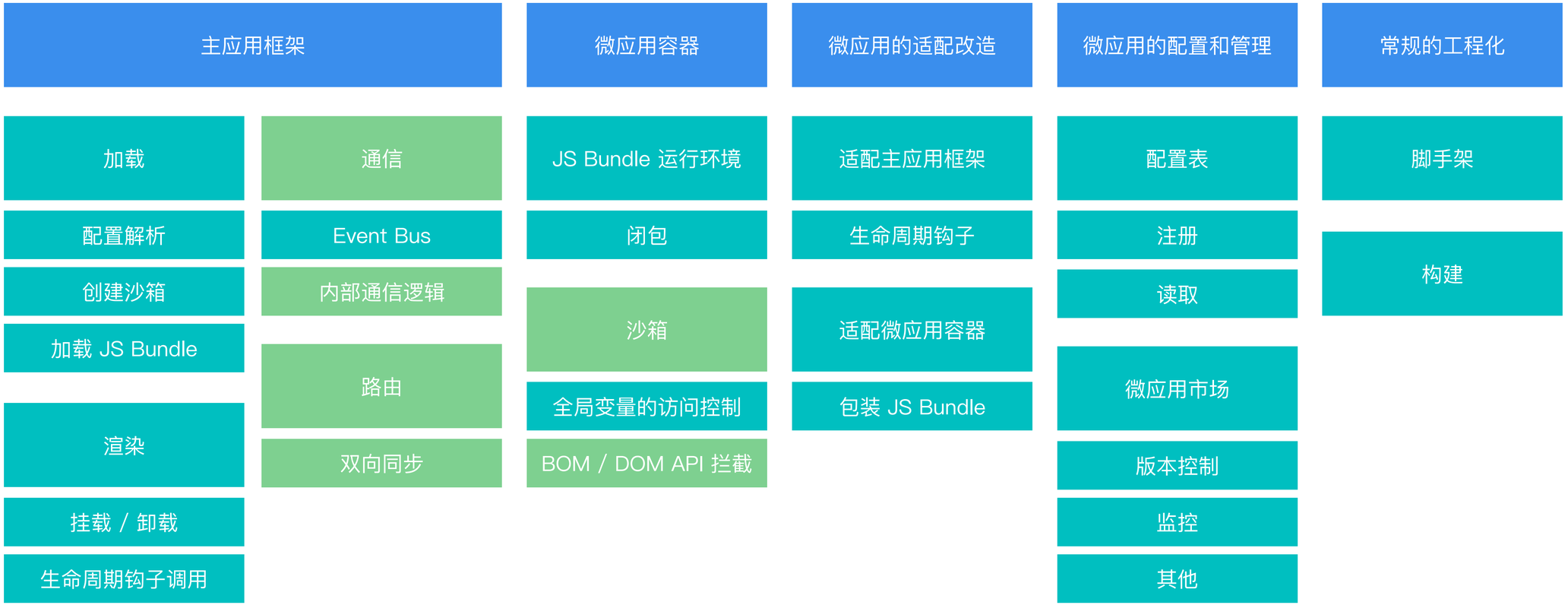




围绕微前端展开的技术体系



微前端内核部分功能模块展开



路由调度问题

解法 a -> 问题 B -> 解法 b -> 问题 C -> ...

点击后退，正确返回上一个页面，且**主应用路由同步**

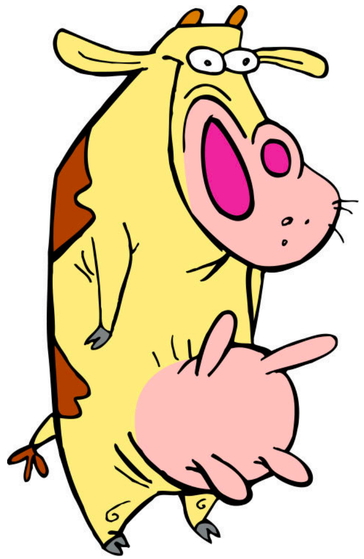
https://cow-and-chicken.com/cow/detail



Cow & Chicken

cow

chicken



七岁，鸡的妹妹，家中的长女。身高约2米、体重约180公斤（600磅）。

虽然已经有成熟牛科动物的外观，但仍保有小女孩的天真无邪，也有些神经质。

会无私地对待家人，热情的奉献自己。

有时候会以超级牛的身份现身，变身后会喊口号：“超级牛来拯救咧！”。经常拯救哥哥鸡。会说西班牙语（中文配音则是带有上海腔）。武器是四个奶头，可以旋转喷奶，还能当做探测器用。没穿裤子先生不遗余力地找寻她的真实身份。

挤出来的牛奶非常好喝。

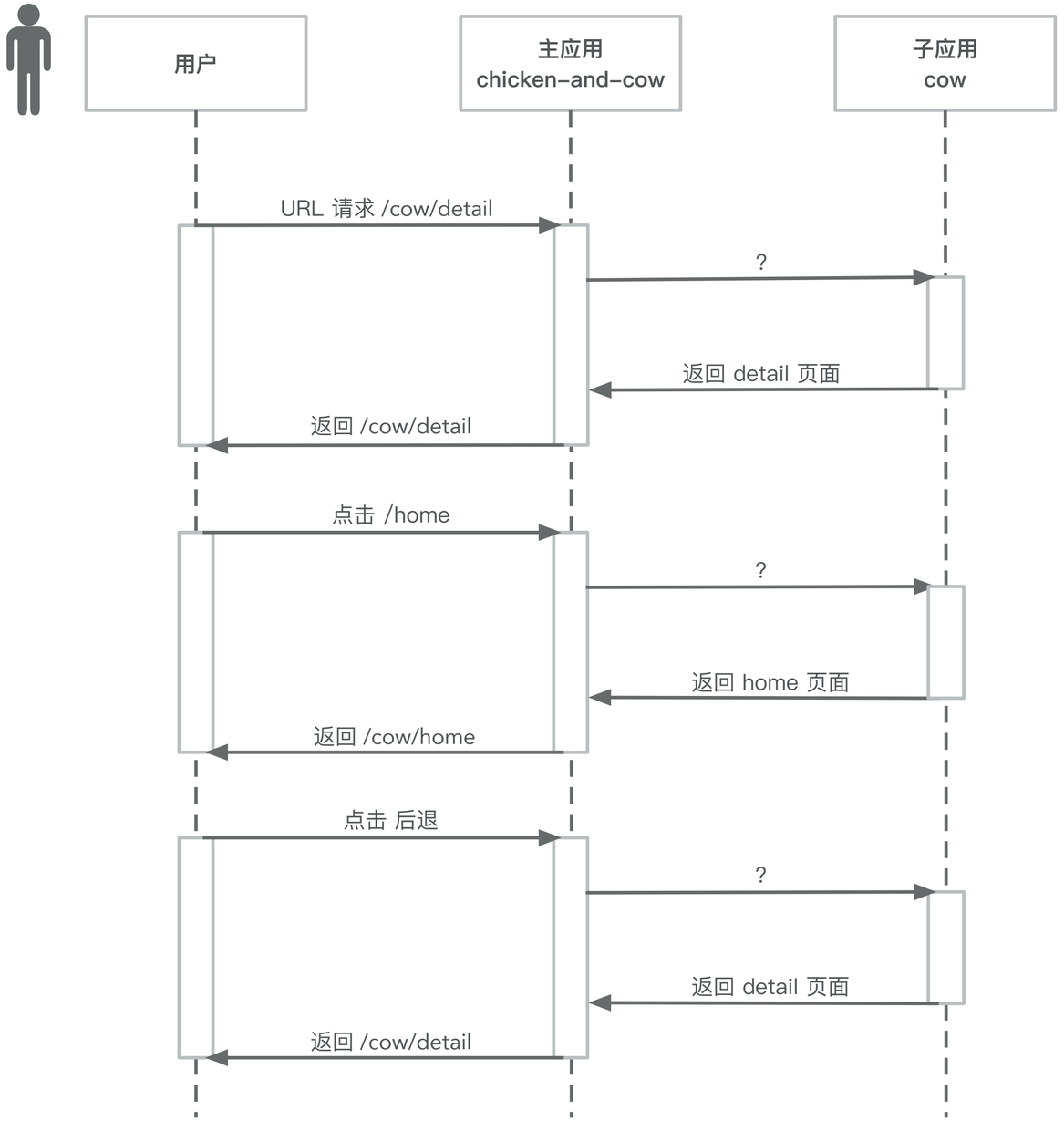
Home

由一个具体的 URL 直达 **目标子应用** 的**对应页面**

点击子应用中的链接，跳转对应页面，且 **主应用路由同步**

达到形如这样的效果

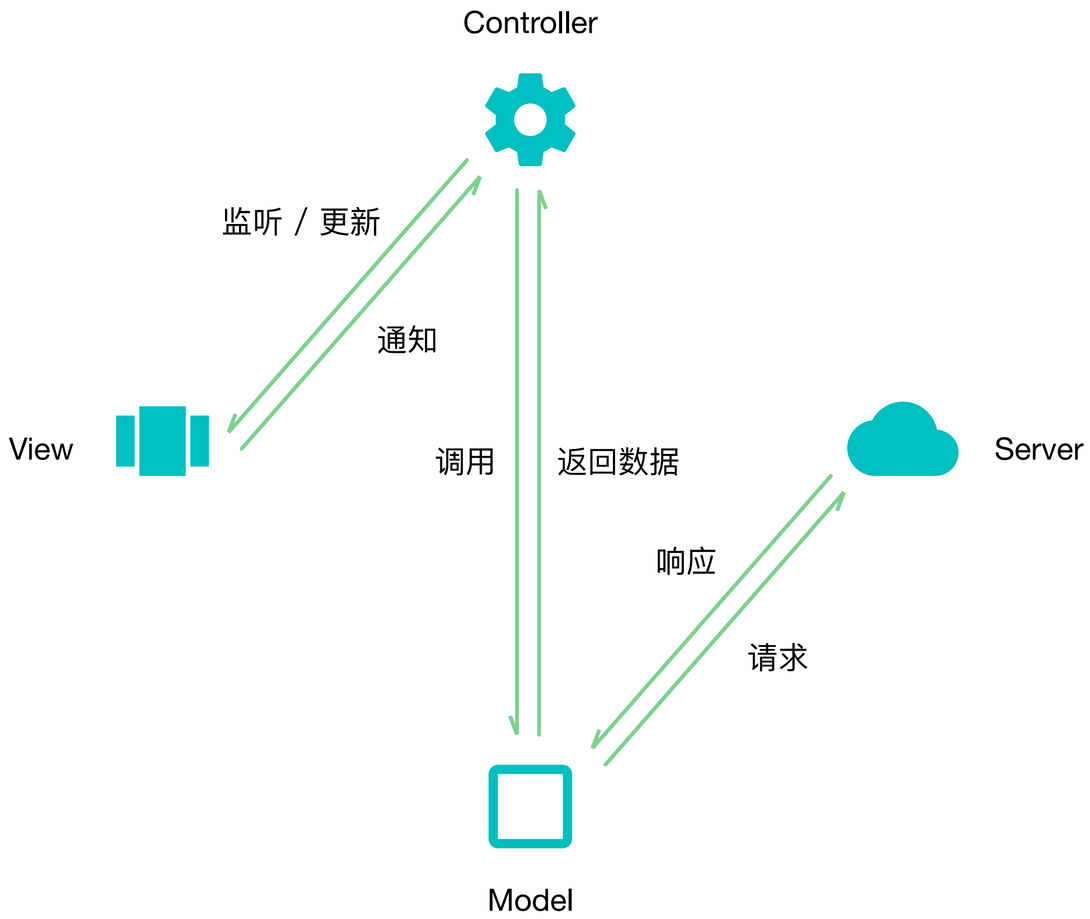
我们需要逐步补全问号部分



且慢！先来谈谈前端路由
……为什么要有前端路由?

在有 Router 之前的年代

Controller 承担了太多




```
// 维护当前的页面状态
var currentState = 'default';

// 类似于今天的路由表
var stateMap = {
  'default': () => {},
  'cow/home': () => {},
  'chicken/home': () => {}
};

// controller 是个 EventEmitter
controller.on('statechange', (e) => {
  stateMap[e.type](e.payload);
});

// 绑定页面的物理事件
document.on('click', (e) => {
  switch(e.target.className) {
    case 'cow-home-button':
      currentState = 'cow/home';
      break;
  }
  controller.emit('statechange', {
    type: currentState,
    payload: {}
  });
});
```

在管理页面状态这件事情上……

Controller

在前端模拟传统
后端的实践



巧妙地利用
Router 来取代
Controller

HISTORY API

管理页面状态的
标准化

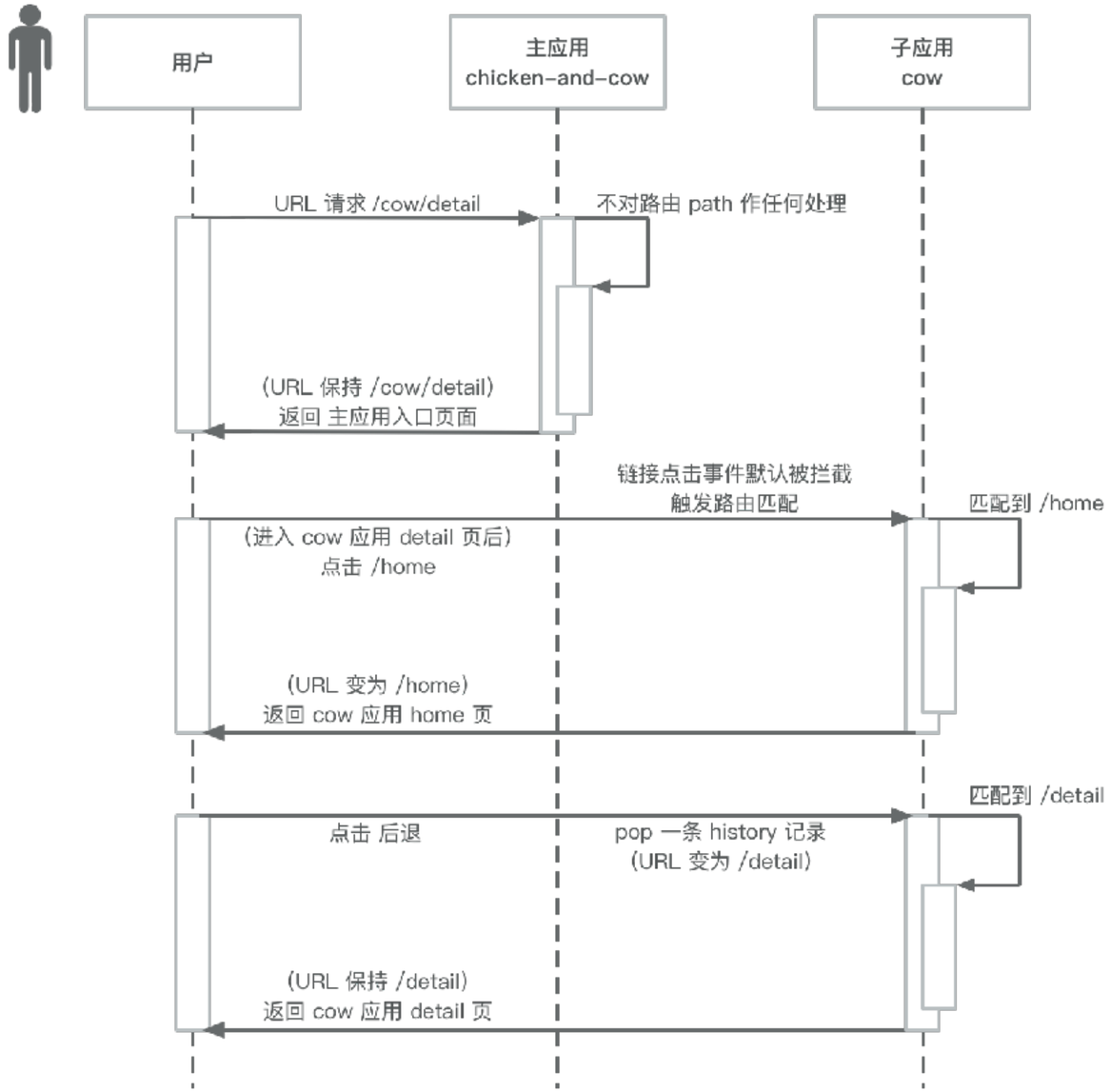


集大成者

主应用完全不处理路由

- 主应用不根据路由调度子应用，子应用内部的状态变化按照原样反映到主应用路由。

- 此方案的问题：路由冲突；初始化无法识别路由。

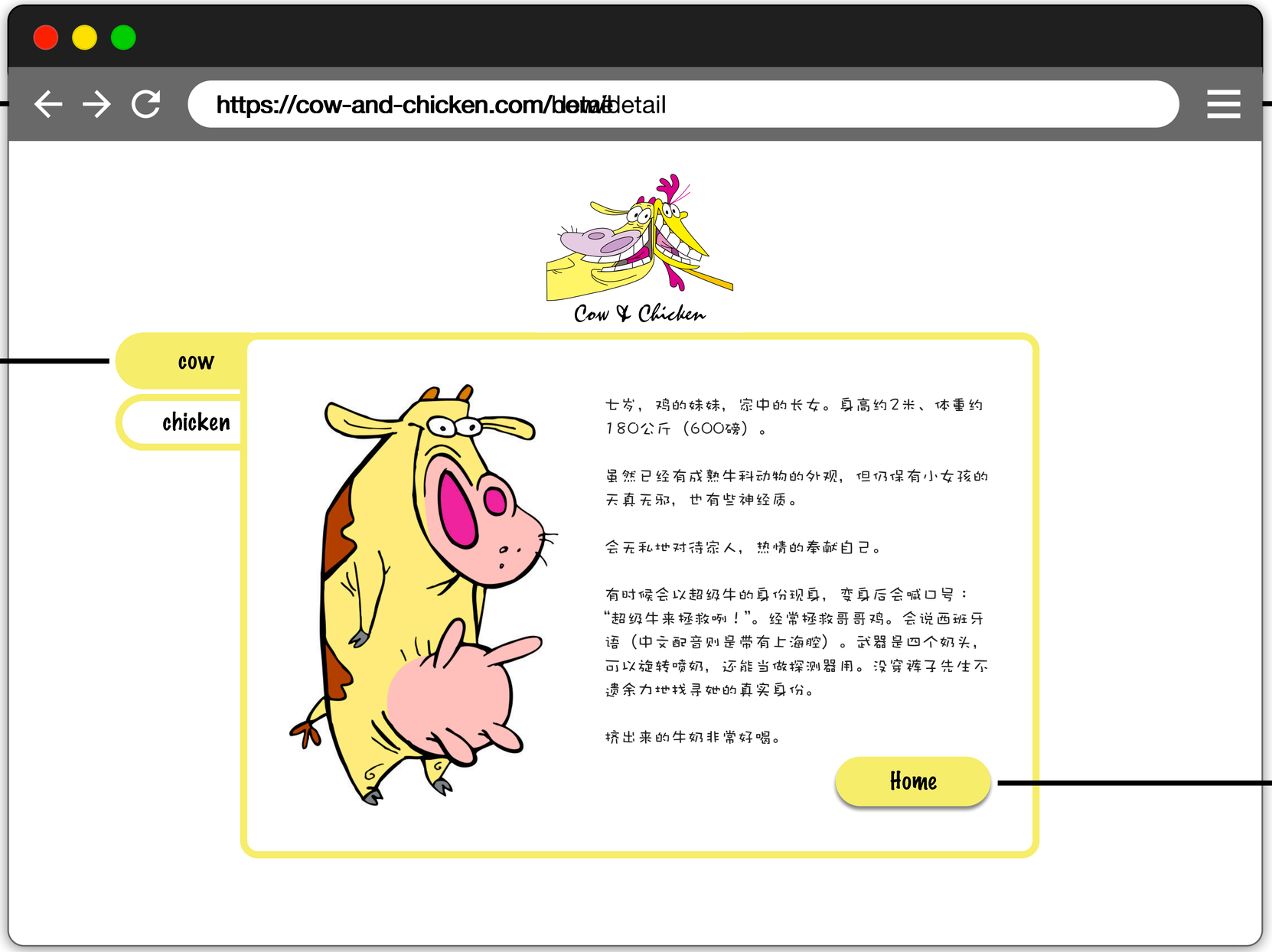


点击后退，返回上一个页面，主应用 URL 同样变化

主应用中点击 cow 标签，进入子应用逻辑，跳转对应页面，同时影响到主应用 URL

由一个具体的 URL 只能进入主应用的首页

点击子应用中的链接，跳转对应页面，同时影响到主应用 URL



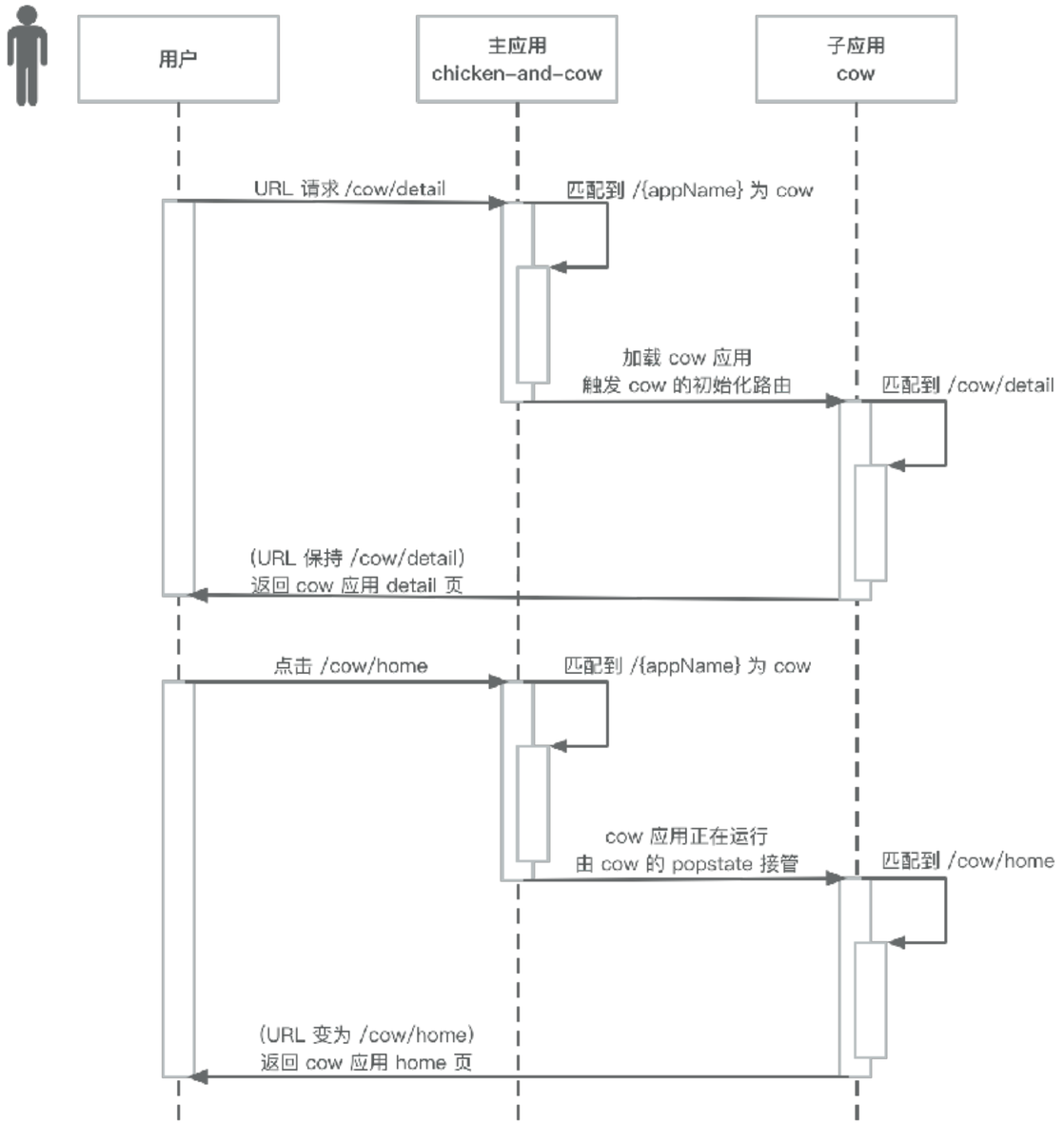
子应用和主应用共享路由

- 主应用根据路由调度子应用，子应用内部的状态变化根据特定规则反映到主应用的路由。

```
// 使用子应用名称来对共享资源切片
const APP_NAME = 'cow';

<Route path={`/${APP_NAME}/home`} component={Home} />
<Route path={`/${APP_NAME}/detail`} component={Detail} />
```

- 上个方案的问题解决了吗?
- 新的问题：需要将约定侵入子应用的逻辑。



点击后退，同样触发子应用的路由切换

https://cow-and-chicken.com/cow/detail

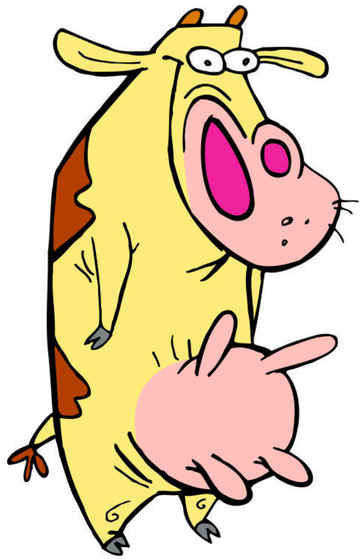
一个具体的 URL 由主应用路由到对应的子应用，然后由子应用接管



Cow & Chicken

cow

chicken



七岁，鸡的妹妹，家中的长女。身高约2米、体重约180公斤（600磅）。

虽然已经有成熟牛科动物的外观，但仍保有小女孩的天真无邪，也有些神经质。

会无私地对待家人，热情的奉献自己。

有时候会以超级牛的身份现身，变身后会喊口号：“超级牛来拯救咧！”。经常拯救哥哥鸡。会说西班牙语（中文配音则是带有上海腔）。武器是四个奶头，可以旋转喷奶，还能当做探测器用。没穿裤子先生不遗余力地找寻她的真实身份。

挤出来的牛奶非常好喝。

Home

点击子应用中的链接，跳转对应页面，主应用路由就是子应用路由

提到「侵入」，想到「隔离」

1. 隔离对全局上下文产生的副作用

- Closure
- Node.js Module

```
// Node.js 中的「The Module Wrapper」
(function(exports, require, module, __filename, __dirname) {
  // 实际的 Module 代码
});

// 子应用被包上 wrapper
__CONSOLE_OS_GLOBAL_HOOK(id, function(exports, require,
module, {
  window
}) {
  // 实际的子应用代码
});
```

2. 特别地，隔离 DOM / BOM 对象

- <iframe>
- Proxy

```
// wrapper 的实现
const frame = document.createElement('iframe');
const _window = frame.contentWindow;

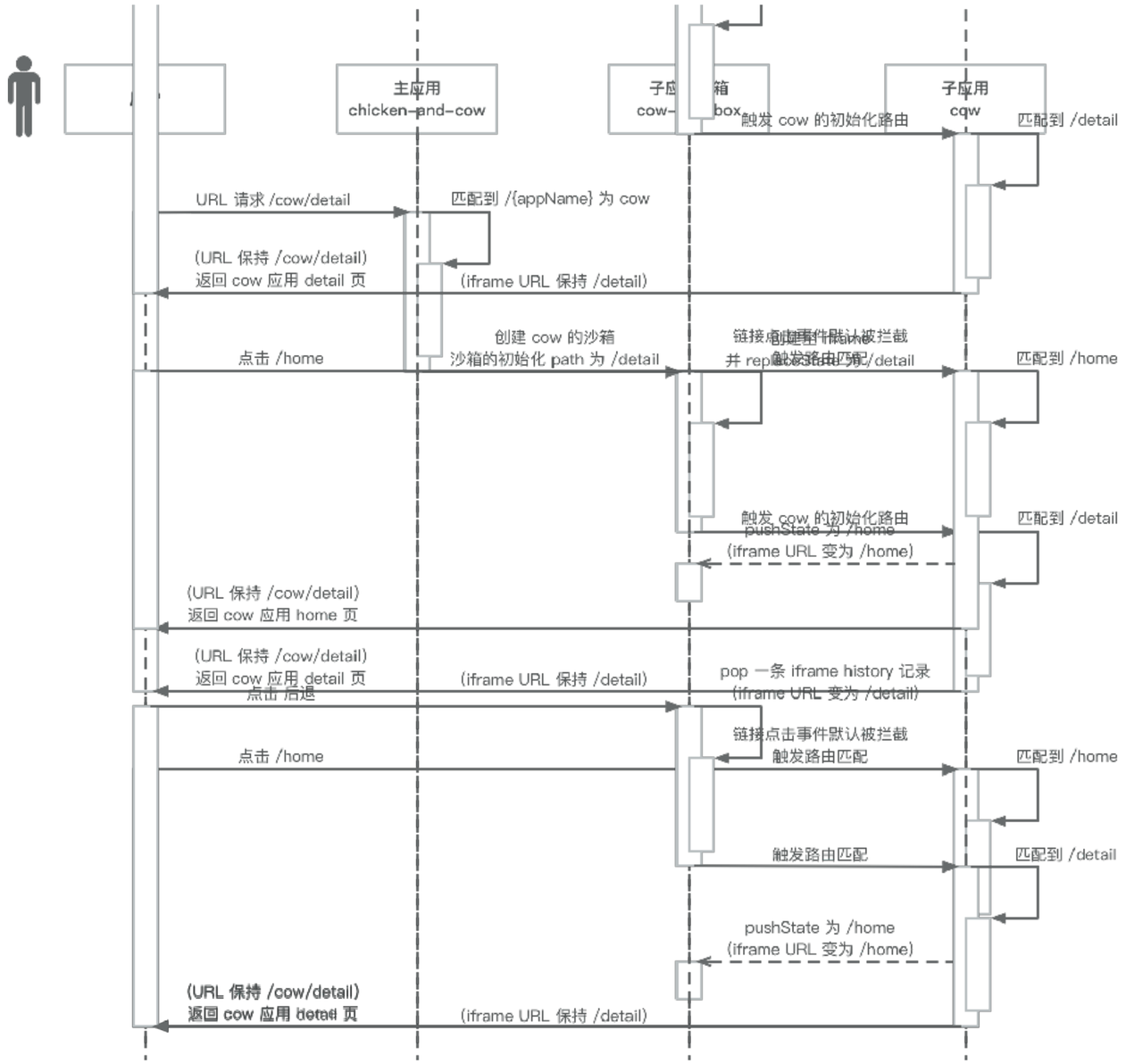
function __CONSOLE_OS_GLOBAL_HOOK(id, entry) {
  entry(exports, require, module, {
    window: _window
  });
}

class History {
  constructor(_history) {
    return new Proxy(_history, {
      get(target, name) {
        switch(name) {
          case 'pushState':
            // 在这里魔改掉
            break;
        }
      }
    });
  }
}
```

子应用维护隔离的路由

- 主应用根据路由调度子应用，子应用内部的状态变化反映到各自的沙箱中，互不干扰，但不会反映到主应用的路由中。

- 上个方案的问题解决了吗?
- 新的问题：由于无法感知子应用路由的变化，主应用不能体现路由逻辑。



点击后退，同样触发子应用（沙箱）的路由切换，主应用路由无变化

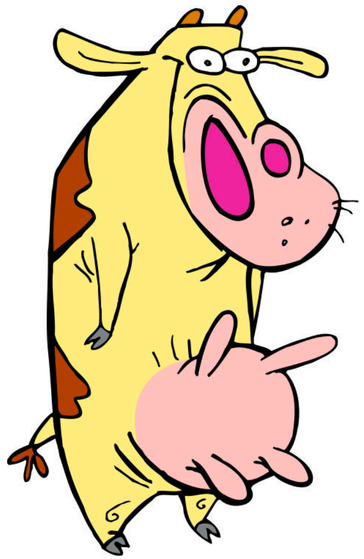
https://cow-and-chicken.com/cow/detail



Cow & Chicken

cow

chicken



七岁，鸡的妹妹，家中的长女。身高约2米、体重约180公斤（600磅）。

虽然已经有成熟牛科动物的外观，但仍保有小女孩的天真无邪，也有些神经质。

会无私地对待家人，热情的奉献自己。

有时候会以超级牛的身份现身，变身后会喊口号："超级牛来拯救咧！"。经常拯救哥哥鸡。会说西班牙语（中文配音则是带有上海腔）。武器是四个奶头，可以旋转喷奶，还能当做探测器用。没穿裤子先生不遗余力地找寻她的真实身份。

挤出来的牛奶非常好喝。

Home

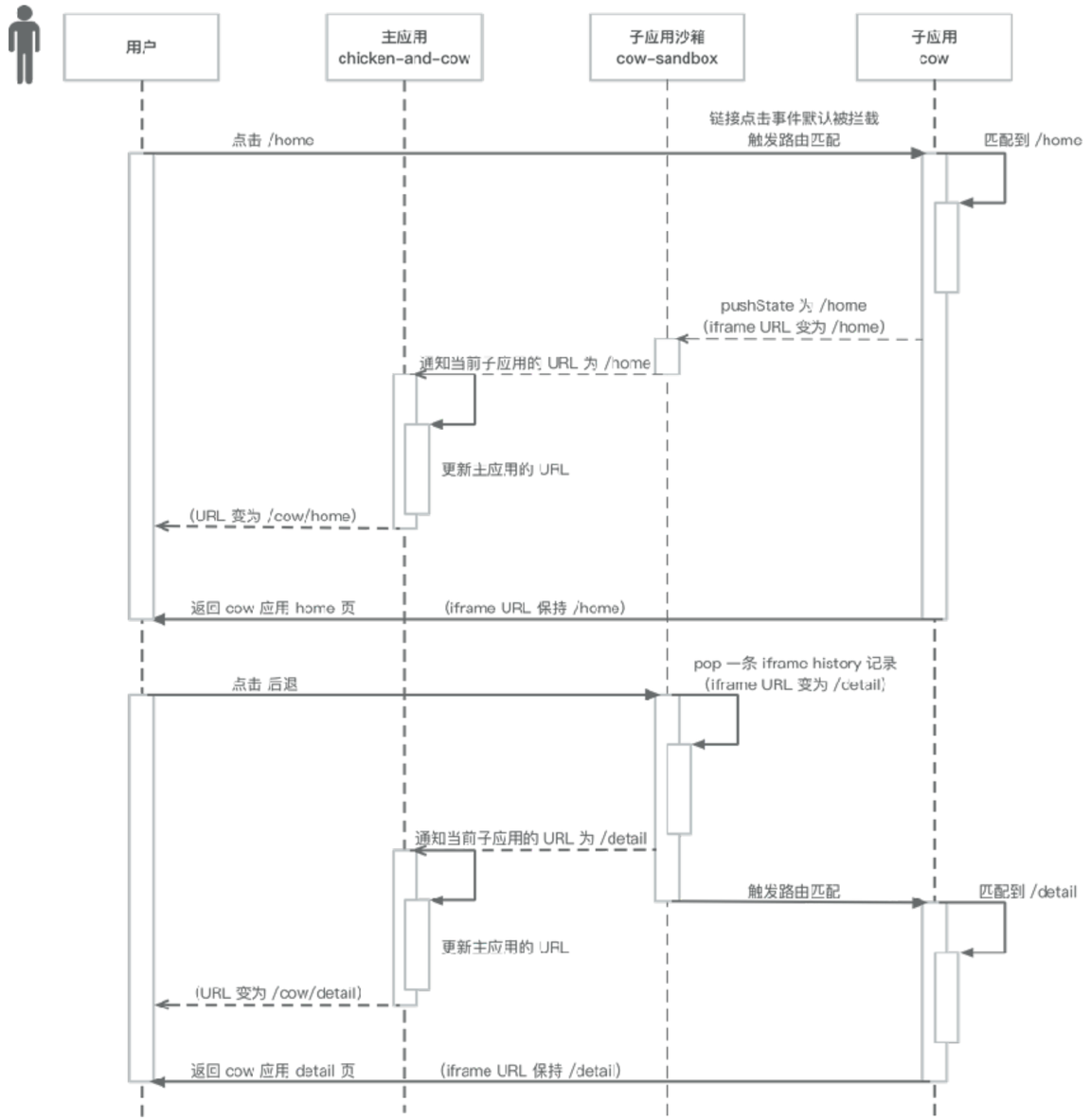
一个具体的 URL 由主应用路由到对应的子应用，然后由子应用接管

点击子应用中的链接，跳转对应页面，主应用路由无变化

子应用路由同步回主应用

- 主应用根据路由调度子应用，子应用内部的状态变化反映到各自的沙箱中，互不干扰，并且最终会反映到主应用的路由中。
- 需要同步？那就通信咯。

- 上个方案的问题解决了吗?
- 新的疑问：
  - 主应用和子应用都改变了 state，这会引起 popstate 无限循环吗?
  - 为何用 replaceState 而非 pushState?



1. 子应用监听路由变化然后通知主应用

- Proxy history
- postMessage

```
switch(name) {
  case 'pushState':
    return (...args) => {
      const returnValue = _pushState.call(_history, ...args);
      // 插入消息通信逻辑
      frame.postMessage({
        type: 'statechange',
        data: {
          location: frame.location,
          state: _history.state
        }
      }, '*');
      return returnValue;
    };
}
```

2. 主应用侧监听此消息

- replaceState

```
frame.contentWindow.addEventListener('message', (e) => {
  const payload = e.data;
  switch(payload.type) {
    case 'statechange':
      const { state, title, location } = payload.data;
      const url = location.href;
      window.history.replaceState(state, title, url);
      break;
  }
});
```

点击后退，正确返回上一个页面，且**主应用路由同步**

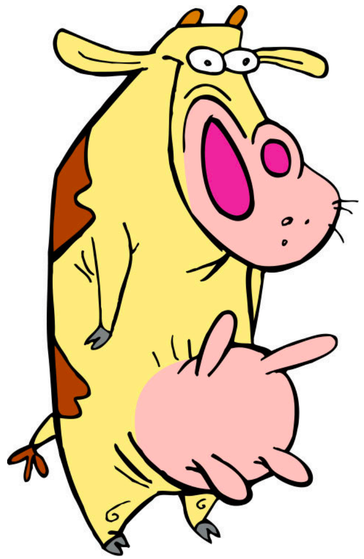
https://cow-and-chicken.com/cow/detail



Cow & Chicken

cow

chicken



七岁，鸡的妹妹，家中的长女。身高约2米、体重约180公斤（600磅）。

虽然已经有成熟牛科动物的外观，但仍保有小女孩的天真无邪，也有些神经质。

会无私地对待家人，热情的奉献自己。

有时候会以超级牛的身份现身，变身后会喊口号：“超级牛来拯救咧！”。经常拯救哥哥鸡。会说西班牙语（中文配音则是带有上海腔）。武器是四个奶头，可以旋转喷奶，还能当做探测器用。没穿裤子先生不遗余力地找寻她的真实身份。

挤出来的牛奶非常好喝。

Home

由一个具体的 URL 直达 **目标子应用** 的**对应页面**

点击子应用中的链接，跳转对应页面，且 **主应用路由同步**

当一个页面中的 iframe 跳转后，parent 页面的地址栏、history 会如何变化?

Joint Session History

The joint session history of a History object is the union of all the session histories of all browsing contexts of all the fully active Document objects that share the History object's top-level browsing context, with all the entries that are current entries in their respective session histories removed except for the current entry of the joint session history.

	行为	主应用	子应用
不干涉	子应用从 /detail 点击到 /home	URL 不变化	页面变为 home
	点击后退	URL 不变化	页面变为 detail
pushState	子应用从 /detail 点击到 /home	URL 变为 /cow/home	页面变为 home
	点击后退	URL 变为 /cow/detail	页面不变化
	点击后退	URL 不变化	页面变为 detail
replaceState	子应用从 /detail 点击到 /home	URL 变为 /cow/home	页面变为 home
	点击后退	URL 变为 /cow/detail	页面变为 detail

回顾

解题思路

1. 理解前端路由的存在意义，明确路由同步的确是个问题。
2. 从最干净自然的处理方式开始，挖掘出方案背后的问题。
3. 提出针对问题的解法，代入；考察是否出现新问题。
4. 如有，回到 3；否则，结束。
5. 顺便在过程中彻底理解一些现象。



谢谢！

- 推荐书：《守望者（Watchmen）》
- 推荐关注：
  - https://github.com/aliyun/alibabacloud-console-os
  - https://github.com/aliyun/alibabacloud-console-widget
  - https://github.com/aliyun/alibabacloud-console-components
- 推荐投递：fahai@taobao.com

